Reinforcement Learning

P Satya Jayadev

Web Enabled M.Tech,
Indian Institute of Technology Madras

Topic 2

Monte Carlo Methods

Outline

1. Intro to MC Methods in RL
2. MC Policy Evaluation
3. MC Control
4. On-Policy and Off-Policy Learning
5. Incremental Monte-Carlo Updates

References: Sutton, R. S., Barto, A. G. (2018). Reinforcement Learning: An Introduction. The MIT Press.

Recall: Value Functions and Bellman Equations

- Optimal State Value:

$$v_*(s) = \max_a \sum_{s',r} p(s', r | s, a) \left[r + \gamma v_*(s') \right]$$

- Optimal Action Value:

$$q_*(s, a) = \sum_{s',r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$

- Optimal Policy: Policy π^* which maximises the expected return in an agent-environment interaction

$$\pi_*(a|s) = \arg\max_a q_*(s, a)$$

- Solving the Bellman optimality equation gives the optimal value function and therefore, an optimal policy

Monte Carlo Methods

Intro to Monte Carlo (MC) Methods in RL

- MC methods learn values from episodes of experience (simulated or actual agent-env interactions)
- MC methods are model-free i,e, no knowledge of state transitions is required
- MC methods can be applied only to episodic tasks

Intro to Monte Carlo (MC) Methods in RL

- MC methods learn values from episodes of experience (simulated or actual agent-env interactions)
- MC methods are model-free i,e, no knowledge of state transitions is required
- MC methods can be applied only to episodic tasks
- Basic Principle: Value $\approx$ Mean return i.e., empirical mean return instead of expected return

Intro to Monte Carlo (MC) Methods in RL

- MC methods learn values from episodes of experience (simulated or actual agent-env interactions)
- MC methods are model-free i,e, no knowledge of state transitions is required
- MC methods can be applied only to episodic tasks
- Basic Principle: Value $\approx$ Mean return i.e., empirical mean return instead of expected return
- Steps in MC Approach to Solving MDPs:
  1. **MC Estimation or Policy Evaluation:** Estimate the state values and action values based on given policy
  2. **MC Control:** Update policy towards optimality based on estimated values

MC Estimation of State Values (Policy evaluation)

- Approach:
  - Sample multiple episodes by following a policy π
  - For a state s, compute the return G_t by adding rewards from the step t when s is visited in an episode
  - Compute average return based on all episodes in which s appears

MC Estimation of State Values (Policy evaluation)

- Approach:
  - Sample multiple episodes by following a policy π
  - For a state s, compute the return G_t by adding rewards from the step t when s is visited in an episode
  - Compute average return based on all episodes in which s appears
- Since s can appear multiple times in an episode, two approaches can be used
  1. First Visit MC state estimation
  2. Every visit MC state estimation

Algorithm: First Visit MC Estimation of State Values

1. **Input:** Policy π to be evaluated
2. **Initialise:** $v_\pi(s) = 0,\ N(s) = 0,\ G(s) = 0$
3. **Loop:**
   3.1 Sample episodes following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$

Algorithm: First Visit MC Estimation of State Values

1. **Input:** Policy π to be evaluated
2. **Initialise:** $v_\pi(s) = 0,\ N(s) = 0,\ G(s) = 0$
3. **Loop:**
   - 3.1 Sample episodes following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$
   - 3.2 For each time step t until T:
   - 3.3 If this is the first time t that state s is visited in the episode:
     - Increment counter of total first visits: $N(s) = N(s) + 1$
     - Compute $G_t = R_t + \gamma R_{t+1} + \ldots$
     - Increment total return: $G(s) = G(s) + G_t$
   - 3.4 Estimated $\hat{v}_\pi(s) = G(s)/N(s)$

Algorithm: First Visit MC Estimation of State Values

1. **Input:** Policy π to be evaluated
2. **Initialise:** $v_\pi(s) = 0,\ N(s) = 0,\ G(s) = 0$
3. **Loop:**
   - 3.1 Sample episodes following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$
   - 3.2 For each time step t until T:
   - 3.3 If this is the first time t that state s is visited in the episode:
     - Increment counter of total first visits: $N(s) = N(s) + 1$
     - Compute $G_t = R_t + \gamma R_{t+1} + \ldots$
     - Increment total return: $G(s) = G(s) + G_t$
   - 3.4 Estimated $\hat{v}_\pi(s) = G(s)/N(s)$
4. By law of large numbers, $\hat{v}_\pi(s) \to v_\pi(s)$ as $N \to \infty$

Algorithm: Every Visit MC Estimation of State Values

1. **Input:** Policy π to be evaluated
2. **Initialise:** $v_\pi(s) = 0,\ N(s) = 0,\ G(s) = 0$
3. **Loop:**
   3.1 Sample episodes following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$
   3.2 For each time step t until T:

Algorithm: Every Visit MC Estimation of State Values

1. **Input:** Policy π to be evaluated
2. **Initialise:** $v_\pi(s) = 0,\ N(s) = 0,\ G(s) = 0$
3. **Loop:**
   - 3.1 Sample episodes following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$
   - 3.2 For each time step t until T:
   - 3.3 Every time state s is visited in time step t in the episode:
     - Increment counter of total first visits: $N(s) = N(s) + 1$
     - Compute $G_t = R_t + \gamma R_{t+1} + \ldots$
     - Increment total return: $G(s) = G(s) + G_t$
   - 3.4 Estimated $\hat{v}_\pi(s) = G(s)/N(s)$
4. By law of large numbers, $\hat{v}_\pi(s) \to v_\pi(s)$ as $N \to \infty$

Running Example: Robot Navigation with a Loop

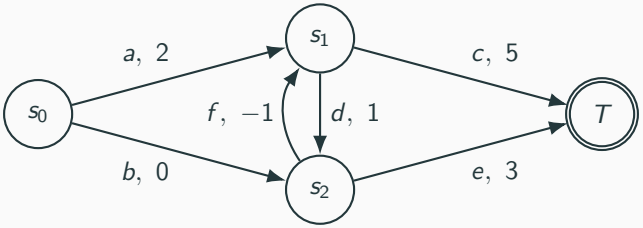
- We continue with a small robot navigation example.
- The robot starts at s_0 and tries to reach terminal state T.
- The robot may pass through s_1 and S_2.
- In some episodes, the robot can return to S_1 again.
- This allows us to compare **First-Visit MC** and **Every-Visit MC**.

$$\gamma = 0.9$$

Running Example: Robot Navigation with a Loop

- We continue with a small robot navigation example.
- The robot starts at s_0 and tries to reach terminal state T.
- The robot may pass through s_1 and S_2.
- In some episodes, the robot can return to S_1 again.
- This allows us to compare **First-Visit MC** and **Every-Visit MC**.

$$\gamma = 0.9$$



Observed Episodes Under a Policy π

- In Monte Carlo methods, we do not need the full model of the environment.
- We observe complete episodes generated by following a policy π.

Observed Episodes Under a Policy π

- In Monte Carlo methods, we do not need the full model of the environment.
- We observe complete episodes generated by following a policy π.

Episode	Observed trajectory
Episode 1	$s_0 \xrightarrow{a,2} s_1 \xrightarrow{d,1} s_2 \xrightarrow{f,-1} s_1 \xrightarrow{c,5} T$
Episode 2	$s_0 \xrightarrow{b,0} s_2 \xrightarrow{e,3} T$
Episode 3	$s_0 \xrightarrow{a,2} s_1 \xrightarrow{c,5} T$

MC Principle
Estimate values by averaging returns observed in complete episodes.

Return from a State Visit

- In Monte Carlo methods, value is estimated using the return after a visit.
- The return from time t is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Return from a State Visit

- In Monte Carlo methods, value is estimated using the return after a visit.
- The return from time t is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Consider Episode 1:

$$s_0 \xrightarrow{2} s_1 \xrightarrow{1} s_2 \xrightarrow{-1} s_1 \xrightarrow{5} T$$

- s_1 is visited twice in this episode.
- First visit to s_1: rewards after visit are $1,\ -1,\ 5$.
- Second visit to s_1: reward after visit is 5.

Returns for Visits to s_1

- We compute returns for each visit to s_1.
- Discount factor: $\gamma = 0.9$.

Returns for Visits to s_1

- We compute returns for each visit to s_1.
- Discount factor: $\gamma = 0.9$.

Episode	**Visit to s_1**	**Return**
Episode 1	First visit	$1 + 0.9(-1) + 0.9^2(5) = 4.15$
Episode 1	Second visit	5
Episode 3	First visit	5

Returns for Visits to s_1

- We compute returns for each visit to s_1.
- Discount factor: $\gamma = 0.9$.

Episode	**Visit to s_1**	**Return**
Episode 1	First visit	$1 + 0.9(-1) + 0.9^2(5) = 4.15$
Episode 1	Second visit	5
Episode 3	First visit	5

Observation
Episode 1 gives two different returns for the same state s_1, because s_1 is visited twice.

First-Visit MC Estimate of $V^{\pi}(s_1)$

- First-Visit MC uses only the **first occurrence** of a state in each episode.
- For s_1, we use:

First-Visit MC Estimate of $V^{\pi}(s_1)$

- First-Visit MC uses only the **first occurrence** of a state in each episode.
- For s_1, we use:

Episode	First-Visit Return for s_1
Episode 1	4.15
Episode 3	5.00

$$V^{\pi}(s_1) = \frac{4.15 + 5.00}{2}$$

$$\boxed{V^{\pi}(s_1) = 4.575}$$

Interpretation
First-Visit MC takes one return sample per episode for a given state.

Every-Visit MC Estimate of $V^{\pi}(s_1)$

- Every-Visit MC uses **every occurrence** of a state in all episodes.
- For s_1, we use:

Every-Visit MC Estimate of $V^{\pi}(s_1)$

- Every-Visit MC uses **every occurrence** of a state in all episodes.
- For s_1, we use:

Episode	Visit	Return
Episode 1	First visit	4.15
Episode 1	Second visit	5.00
Episode 3	First visit	5.00

Every-Visit MC Estimate of $V^{\pi}(s_1)$

- Every-Visit MC uses **every occurrence** of a state in all episodes.
- For s_1, we use:

Episode	Visit	Return
Episode 1	First visit	4.15
Episode 1	Second visit	5.00
Episode 3	First visit	5.00

$$V^{\pi}(s_1) = \frac{4.15 + 5.00 + 5.00}{3}$$

$$\boxed{V^{\pi}(s_1) = 4.7167}$$

Interpretation
Every-Visit MC treats every visit as a separate sample of return.

MC Estimation of Action Values

- With a model, state values are sufficient to determine an optimal policy
- Without a model, actions values are to be estimated to determine an optimal policy

MC Estimation of Action Values

- With a model, state values are sufficient to determine an optimal policy
- Without a model, actions values are to be estimated to determine an optimal policy
- Same approach as state value estimation
- States visits are replaced by state-action visits
- Issue: If π is a deterministic policy, then same action is visited in each state in every episode

MC Estimation of Action Values

- With a model, state values are sufficient to determine an optimal policy
- Without a model, actions values are to be estimated to determine an optimal policy
- Same approach as state value estimation
- States visits are replaced by state-action visits
- Issue: If π is a deterministic policy, then same action is visited in each state in every episode
- Two workarounds:
  1. Exploring starts with random state-action pairs
  2. Exploring the environment with a stochastic policy

MC Estimation of Action Values

- MC estimation can also be used for action values.
- Instead of state visits, we look at **state-action visits**.
- Let us estimate:

$$Q^{\pi}(s_1, d)$$

- This asks: what is the return after taking action d from state s_1?

MC Estimation of Action Values

- MC estimation can also be used for action values.
- Instead of state visits, we look at **state-action visits**.
- Let us estimate:

$$Q^{\pi}(s_1, d)$$

- This asks: what is the return after taking action d from state s_1?

In Episode 1:

$$s_1 \xrightarrow{d,1} s_2 \xrightarrow{f,-1} s_1 \xrightarrow{c,5} T$$

$$G = 1 + 0.9(-1) + 0.9^2(5) = 4.15$$

Action-Value Estimate for $Q^{\pi}(s_1, d)$

- Suppose we later observe one more episode:

$$s_0 \xrightarrow{a,2} s_1 \xrightarrow{d,1} s_2 \xrightarrow{e,3} T$$

The return after taking (s_1, d) in this episode is:

$$G = 1 + 0.9(3) = 3.7$$

Action-Value Estimate for $Q^{\pi}(s_1, d)$

- Suppose we later observe one more episode:

$$s_0 \xrightarrow{a,2} s_1 \xrightarrow{d,1} s_2 \xrightarrow{e,3} T$$

The return after taking (s_1, d) in this episode is:

$$G = 1 + 0.9(3) = 3.7$$

Now we have two return samples for (s_1, d) and $Q^{\pi}(s_1, d)$ is estimated as:

$$\hat{Q}^{\pi}(s_1, d) = \frac{4.15 + 3.70}{2}$$

$$\boxed{\hat{Q}^{\pi}(s_1, d) = 3.925}$$

Interpretation
$Q^{\pi}(s_1, d)$ estimates how good it is to take action d from state s_1 and then follow policy π.

Incremental Monte Carlo Updates

- An approach to update the estimates of state or action values incrementally after each episode
- Suppose $V_n(s)$ and $Q_n(s,a)$ are state and action value estimates after n updates and G is the latest return
- Formula for MC Incremental Update:

$$V_{n+1}(s) = V_n(s) + \alpha(G - V_n(s))$$
$$Q_{n+1}(s) = Q_n(s,a) + \alpha(G - Q_n(s,a))$$

- α is called learning rate and can be fixed or taken as $1/n$

Incremental Monte Carlo Updates

- An approach to update the estimates of state or action values incrementally after each episode
- Suppose $V_n(s)$ and $Q_n(s,a)$ are state and action value estimates after n updates and G is the latest return
- Formula for MC Incremental Update:

$$V_{n+1}(s) = V_n(s) + \alpha(G - V_n(s))$$
$$Q_{n+1}(s) = Q_n(s,a) + \alpha(G - Q_n(s,a))$$

- α is called learning rate and can be fixed or taken as $1/n$
- Interpretation:
  - $(G - V_n(s))$ can be seen as error between observed return (target) and value estimate
  - Value is updated iteratively by a fraction of error leading to reduced error

Incremental First-Visit MC for $V(s_1)$

- First-Visit returns for s_1: $4.15, 5$
- Assumed initial estimate:

$$V_0(s_1) = 0$$

- Use sample-average step size:

$$\alpha_n = \frac{1}{n}$$

Incremental First-Visit MC for $V(s_1)$

- First-Visit returns for s_1: $4.15, 5$
- Assumed initial estimate:

$$V_0(s_1) = 0$$

- Use sample-average step size:

$$\alpha_n = \frac{1}{n}$$

First update:

$$V_1(s_1) = V_0(s_1) + \alpha(G - V_0(s_1)) = V_0(s_1) + 1(4.15 - 0)$$

$$\boxed{V_1(s_1) = 4.15}$$

Incremental First-Visit MC for $V(s_1)$

- First-Visit returns for s_1: $4.15, 5$
- Assumed initial estimate:

$$V_0(s_1) = 0$$

- Use sample-average step size:

$$\alpha_n = \frac{1}{n}$$

First update:

$$V_1(s_1) = V_0(s_1) + \alpha(G - V_0(s_1)) = V_0(s_1) + 1(4.15 - 0)$$

$$\boxed{V_1(s_1) = 4.15}$$

Second update:

$$V_2(s_1) = V_1(s_1) + \alpha(G - V_1(s_1)) = 4.15 + \frac{1}{2}(5.00 - 4.15)$$

$$\boxed{V_2(s_1) = 4.575}$$

MC Control

- Policy evaluation and policy improvement are alternately performed until optimality is achieved

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_1 \ldots \xrightarrow{I} \pi_*$$

MC Control

- Policy evaluation and policy improvement are alternately performed until optimality is achieved

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_1 \ldots \xrightarrow{I} \pi_*$$

- Policy Improvement: Greedy policy w.r.t. current action value function

$$\pi(s) = \arg\max_a q(s, a)$$

- Policy improvement theorem ensures that this policy converges to optimal policy
- However, policy improvement after complete policy evaluation is impractical

MC Control

- Policy evaluation and policy improvement are alternately performed until optimality is achieved

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_1 \ldots \xrightarrow{I} \pi_*$$

- Policy Improvement: Greedy policy w.r.t. current action value function

$$\pi(s) = \arg\max_a q(s, a)$$

- Policy improvement theorem ensures that this policy converges to optimal policy
- However, policy improvement after complete policy evaluation is impractical
- Practical Approach: Alternate between policy evaluation and policy improvement after every episode of experience to reasonably converge with limited episodes

On-Policy and Off-Policy Learning

On-Policy and Off-Policy Learning

- On-Policy learning:
  - Evaluate and improve the policy that is currently being used to make decisions
  - Exploration needs to be carefully managed
  - Learning is more stable and better chance of converge

On-Policy and Off-Policy Learning

- On-Policy learning:
  - Evaluate and improve the policy that is currently being used to make decisions
  - Exploration needs to be carefully managed
  - Learning is more stable and better chance of converge
- Off-Policy learning:
  - Evaluate and improve a policy (Target policy) that is different from the one (Behaviour policy) used to interact with the environment
  - Exploration and exploitation are separated and there is more flexibility
  - Less stable and may result in divergence is some cases
- In MC Methods, on-Policy and off-policy approaches are required to avoid exploring starts

On-Policy MC Method

- In On-Policy MC method, a soft policy is followed to balance between exploration and exploitation
- Soft Policy: A stochastic policy in which all possible actions in a state can be chosen with a non-zero probability

On-Policy MC Method

- In On-Policy MC method, a soft policy is followed to balance between exploration and exploitation
- Soft Policy: A stochastic policy in which all possible actions in a state can be chosen with a non-zero probability
- ϵ – *Greedy* Policy:
  - A soft policy in which random action is chosen with a small probability ϵ (exploration)
  - Greedy action is chosen with probability $1 - \epsilon$ (exploitation)
  - Closest to greedy policy which ensures convergence

Algorithm: On Policy First Visit MC Control

1. **Initialise:**
   - $\epsilon-$greedy policy π with small $\epsilon > 0$
   - $q_\pi(s,a) = 0 \; \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, $N(s,a) = 0$, $G(s,a) = 0$
2. **Loop:**
   - 2.1 Sample an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$
   - 2.2 For each time step t until T:
   - 2.3 If this is the first time t that action a is taken in state s :
     - Increment counter of total first visits: $N(s,a) = N(s,a) + 1$
     - Compute $G_t = R_t + \gamma R_{t+1} + \ldots$
     - Increment total return: $G(s,a) = G(s,a) + G_t$
   - 2.4 Estimated $\hat{q}_\pi(s,a) = G(s,a)/N(s,a)$

Algorithm: On Policy First Visit MC Control

1. **Initialise:**
   - $\epsilon-$greedy policy π with small $\epsilon > 0$
   - $q_\pi(s,a) = 0 \; \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, $N(s,a) = 0$, $G(s,a) = 0$
2. **Loop:**
   - 2.1 Sample an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$
   - 2.2 For each time step t until T:
   - 2.3 If this is the first time t that action a is taken in state s :
     - Increment counter of total first visits: $N(s,a) = N(s,a) + 1$
     - Compute $G_t = R_t + \gamma R_{t+1} + \ldots$
     - Increment total return: $G(s,a) = G(s,a) + G_t$
   - 2.4 Estimated $\hat{q}_\pi(s,a) = G(s,a)/N(s,a)$
   - 2.5 Update $a^* = \arg\max_a q_\pi(s,a)$

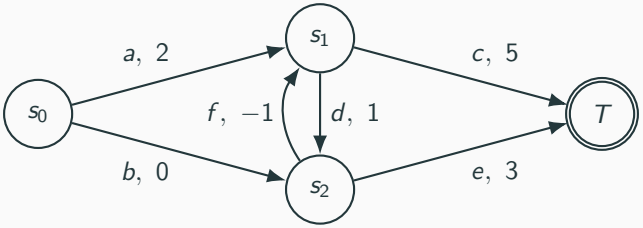
On-Policy First-Visit MC Control: Running Example

- We use the same robot navigation MDP.
- The robot starts at s_0 and eventually reaches terminal state T.
- We want to learn a better policy from sampled episodes.
- Discount factor:

$$\gamma = 0.9$$

- Exploration parameter:

$$\epsilon = 0.2$$



Initial Policy and Initial Action Values

- Assume that initially all action values are zero:

$$Q(S, A) = 0 \quad \text{for all state-action pairs}$$

- Start with an exploratory policy that chooses actions uniformly.

State	Available Actions	Initial Policy π_0
S_0	a, b	$\pi_0(a\|S_0) = 0.5, \quad \pi_0(b\|s_0) = 0.5$
S_1	c, d	$\pi_0(c\|s_1) = 0.5, \quad \pi_0(d\|s_1) = 0.5$
S_2	e, f	$\pi_0(e\|s_2) = 0.5, \quad \pi_0(f\|s_2) = 0.5$

ϵ-Greedy Rule for Two Actions
If one action is greedy, assign probability 0.9 to it and 0.1 to the other action, since $\epsilon = 0.2$.

Iteration 1: Sample an Episode Using π_0

- Suppose the following episode is generated using the initial policy π_0:

$$s_0 \xrightarrow{b,0} s_2 \xrightarrow{f,-1} s_1 \xrightarrow{d,1} s_2 \xrightarrow{e,3} T$$

- We now compute returns for the first visit of each state-action pair.
- Since this is First-Visit MC control, each state-action pair is updated only from its first occurrence in the episode.

State-Action Pair	Rewards After Visit	Return
(s_0, b)	$0, -1, 1, 3$	$0 + 0.9(-1) + 0.9^2(1) + 0.9^3(3) = 2.097$
(s_2, f)	$-1, 1, 3$	$-1 + 0.9(1) + 0.9^2(3) = 2.33$
(s_1, d)	$1, 3$	$1 + 0.9(3) = 3.70$
(s_2, e)	3	3.00

Iteration 1: Update Q and Improve Policy

- Since each state-action pair is seen for the first time, the new Q value is the observed return.

State	Estimated Action Values After Episode 1	Greedy Action
s_0	$Q(s_0, a) = 0, \quad Q(s_0, b) = 2.097$	b
s_1	$Q(s_1, c) = 0, \quad Q(s_1, d) = 3.70$	d
s_2	$Q(s_2, e) = 3.00, \quad Q(s_2, f) = 2.33$	e

Policy is improved to be ϵ-greedy with respect to the updated Q values:

$$\pi_1(b|s_0) = 0.9, \quad \pi_1(a|s_0) = 0.1$$

$$\pi_1(d|s_1) = 0.9, \quad \pi_1(c|s_1) = 0.1$$

$$\pi_1(e|s_2) = 0.9, \quad \pi_1(f|s_2) = 0.1$$

Iteration 2: Sample an Episode Using π_1

- Now the next episode is generated using the improved policy π_1.
- Because the policy is still exploratory, non-greedy actions can occasionally be tried.

Suppose the sampled episode is:

$$s_0 \xrightarrow{a,2} s_1 \xrightarrow{c,5} T$$

First-visit returns:

$$G(s_0, a) = 2 + 0.9(5) = 6.50$$

$$G(s_1, c) = 5.00$$

Iteration 2: Update Q Values

- The newly observed returns update the corresponding state-action values.
- Assume sample-average First-Visit MC updates.

Before Episode 2:

$$Q(s_0, a) = 0, \quad Q(s_0, b) = 2.097$$
$$Q(s_1, c) = 0, \quad Q(s_1, d) = 3.70$$

Episode 2 gives:

$$G(s_0, a) = 6.50, \qquad G(s_1, c) = 5.00$$

Therefore:

$$Q(s_0, a) = 6.50$$
$$Q(s_1, c) = 5.00$$

Updated action values:

$$Q(s_0, a) = 6.50, \quad Q(s_0, b) = 2.097$$
$$Q(s_1, c) = 5.00, \quad Q(s_1, d) = 3.70$$
$$Q(s_2, e) = 3.00, \quad Q(s_2, f) = 2.33$$

Iteration 2: Policy Improvement

- We again make the policy ϵ-greedy with respect to the updated Q values.

State	Greedy After Iteration 1	Greedy After Iteration 2	Policy Change
s_0	b	a	Switches from b to a
s_1	d	c	Switches from d to c
s_2	e	e	No change

New policy π_2:

$$\pi_2(a|s_0) = 0.9, \quad \pi_2(b|s_0) = 0.1$$

$$\pi_2(c|s_1) = 0.9, \quad \pi_2(d|s_1) = 0.1$$

$$\pi_2(e|s_2) = 0.9, \quad \pi_2(f|s_2) = 0.1$$

Off-Policy MC Method

- Suppose π is a fixed target policy (generally deterministic) and b is a fixed behaviour policy (generally stochastic)
- Assumption: Every action taken under π is at least occasionally taken under b
- Objective in Policy Evaluation: Estimate q_π based on episodes drawn by following policy b

Off-Policy MC Method

- Suppose π is a fixed target policy (generally deterministic) and b is a fixed behaviour policy (generally stochastic)
- Assumption: Every action taken under π is at least occasionally taken under b
- Objective in Policy Evaluation: Estimate q_π based on episodes drawn by following policy b
- Issue:
  - Distribution of a sample trajectory or episode taken under b will be different from that taken under π
  - Sampled returns using policy b are not representative of target policy π

Off-Policy MC Method

- Suppose π is a fixed target policy (generally deterministic) and b is a fixed behaviour policy (generally stochastic)
- Assumption: Every action taken under π is at least occasionally taken under b
- Objective in Policy Evaluation: Estimate q_π based on episodes drawn by following policy b
- Issue:
  - Distribution of a sample trajectory or episode taken under b will be different from that taken under π
  - Sampled returns using policy b are not representative of target policy π
- Importance Sampling to the rescue!

Importance Sampling

- **Recall:**In MC methods, expected values are estimated as empirical means
- Importance Sampling: Technique for estimating expected values under one distribution given samples from another

Importance Sampling

- **Recall:**In MC methods, expected values are estimated as empirical means
- Importance Sampling: Technique for estimating expected values under one distribution given samples from another
- Approach: Weigh the returns from policy b to make them representative of the returns from policy π
- This weighing is done by taking the ratio of probability of occurrence of a trajectory under target policy π to that under behaviour policy b

Importance Sampling

- **Recall:**In MC methods, expected values are estimated as empirical means
- Importance Sampling: Technique for estimating expected values under one distribution given samples from another
- Approach: Weigh the returns from policy b to make them representative of the returns from policy π
- This weighing is done by taking the ratio of probability of occurrence of a trajectory under target policy π to that under behaviour policy b
- Importance Sampling Ratio: For a trajectory $\tau = S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T, R_T$:

$$\rho_{t:T-1} = \prod_{i=t}^{T-1} \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$$

$$v_\pi(s) = \mathbb{E}[\rho_{t:T-1} G_t | S_t = s]$$

Example: Learning to play Blackjack using MC methods

- Goal of the game: Obtain cards whose numerical value is as great as possible without exceeding a value of 21
- Rules of the game:
  - 2-players (dealer vs player), all face cards count as 10 and an ace can be either 1 or 11
  - Game begins with both having two cards with one of the dealers card facing up and the other is down
  - If the player has 21, it is called natural and he wins unless the dealer also has a natural, in which case, it is a draw
  - If the player does not have natural, he can request additional cards, one by one (hits) until he either stops (sticks) or exceeds 21 (goes bust and loses the game)
  - If he sticks, then now it's the dealer's turn to either hit or stick. The one close to goal is the winner.

MDP formulation of Blackjack

- Episodic MDP since there is a defined termination
- State Variables: Two variables: 1) Dealer's faced up card 2) Players current total
- Actions: Two possible actions: 1) Hit (Request an additional card) 2) Stick (End the turn of hit)
- Reward Function:
  1. Win : $+1$
  2. Lose: -1
  3. Draw: 0
- **Exercise:** Write a python code which simulates the blackjack environment and finds an optimal policy using Off-policy and On-policy MC methods

Summary of MC Methods

- MC methods are model-free methods to learn optimal policies from episodes of experience
- In MC methods, expected state or action values are estimated using empirical means
- Estimation of action values is required to determining an optimal policy

Summary of MC Methods

- MC methods are model-free methods to learn optimal policies from episodes of experience
- In MC methods, expected state or action values are estimated using empirical means
- Estimation of action values is required to determining an optimal policy
- MC control includes estimation of action values (policy evaluation) and improving policy based on estimated action values (policy improvement)
- In On-policy methods, the policy evaluated and improved is same as the one used for interaction with the environment
- In Off-policy methods, target policy is improved via importance sampling, based on experienced gained by following behaviour policy